

AHSANULLAH UNIVERSITY OF SCIENCE AND TECHNOLOGY



Department of Computer Science and Engineering

Course No: CSE 2104

Course Title: Data Structures Lab

Assignment No: 05

Date of Submission: 19/08/2026

Submitted by,

Name: Hasibul Hasan

Student ID: 00724205101098

Lab Section: B2

Academic Semester: Fall 2025

Program: BSc in CSE

CSE2104 Offline-5 Report

Tasks / Questions:

Implement the below functions of doubly linked list in a single .cpp file.

- (1) insert first
- (2) insert last
- (3) insert anywhere between two nodes (by position, by value)
- (4) delete first
- (5) delete last
- (6) delete from anywhere between two nodes (by position, by value)
- (7) printingF() and printingB() -> functions to print the linked list
- (8) searching() -> function to search a value in the linked list
- (9) last_node() -> function to print the value of the last node
- (10) previous_of_last_node() -> function to print the value of the previous node of last node
- (11) list_size() -> function to print the size of the linked list
- (12) reversePrint() -> function to print the linked list in reverse order
- (13) main function -> no manual node creation/connection is allowed

Source Code (C++):

```
#include<bits/stdc++.h>
using namespace std;
struct dnode
{
    int data;
    dnode *next;
    dnode *prev;
};
dnode *head=NULL;
dnode *tail=NULL;
int c=0;
void printF()
{
    dnode *k=head;
    while(k!=NULL)
    {
        cout<<k->data<<" ";
        k=k->next;
    }
    cout<<endl;
}
void printB()
{
    dnode* l=tail;
    while(l!=NULL)
    {
        cout<<l->data<<" ";
        l=l->prev;
    }
}
```

```

        cout<<endl;
    }
    void insertFirst(int d)
    {
        dnode *newn=(dnode *)malloc(sizeof(dnode));
        newn->data=d;
        newn->prev = NULL;

        if(head==NULL)
        {
            head=newn;
            newn->next=NULL;
            newn->prev=NULL;
            tail=newn;
        }
        else
        {
            newn->prev=NULL;
            newn->next=head;
            head->prev=newn;
            head=newn;
        }
    }
    void insertLast(int d)
    {
        dnode *newn;
        newn=(dnode*)malloc(sizeof(dnode));
        newn->data=d;
        newn->next=NULL;
        if(head==NULL)
        {
            head=newn;
            newn->prev=NULL;
            newn->next=NULL;
            tail=newn;
        }
        else
        {
            dnode *k=head;
            while(k->next!=NULL)
            {
                k=k->next;
            }
            newn->next=NULL;
            newn->prev=k;
            tail=newn;
            k->next=newn;
        }
    }
    void insertAtPos(int d,int pos)
    {

```

```

dnode *newn;
newn=(dnode*)malloc(sizeof(dnode));
newn->data=d;
if(pos==1)
{
    insertFirst(d);
}
else
{
    dnode *k=head;
    for(int i=1; i<pos && k!=NULL; i++)
    {
        k=k->next;
    }
    if(k->next==NULL)
    {
        insertLast(d);
        return;
    }
    newn->prev=k->prev;
    newn->next=k;
    k->prev->next=newn;

    k->prev=newn;
}
}
void deleteFirst()
{
    if(head==NULL)
    {
        return;
    }
    else if(head->next==NULL)
    {
        free(head);
        head=NULL;
        tail=NULL;
    }
    else
    {
        dnode *k=head;
        head=head->next;
        if(head != NULL)
            head->prev = NULL;
        free(k);
    }
}
void deleteLast()
{
    if(head==NULL)
    {

```

```

        return;
    }
    else if(head->next==NULL)
    {
        free(head);
        head=NULL;
        tail=NULL;
        return;
    }
    else
    {
        dnode *k=head;
        while(k->next!=NULL)
        {
            k=k->next;
        }
        tail=k->prev;

        k->prev->next=NULL;

        free(k);
    }
}

```

```

void deleteByPos(int pos)
{
    if(head==NULL)
    {
        return;
    }
    else if(pos==1)
    {
        deleteFirst();
        return;
    }
    else
    {
        dnode *k=head;
        int i;
        for (i=1; i<pos && k!=NULL; i++)
        {
            k=k->next;
        }
        if(k->next==NULL && i==pos)
        {
            deleteLast();
            return;
        }
        else
        {
            k->prev->next=k->next;

```

```

        k->next->prev=k->prev;

        free(k);
        return;
    }
}

void deleteByValue(int v)
{
    if(head==NULL)
    {
        return;
    }
    else if(head->data==v)
    {
        deleteFirst();
        return;
    }
    else
    {
        dnode *k=head;
        while(k->next!=NULL && k->data!=v)
        {
            k=k->next;
        }

        if(k->next==NULL && k->data==v)
        {
            deleteLast();
            return;
        }
        else if(k->data!=v && k->next==NULL)
        {
            return;
        }
        else
        {
            k->prev->next=k->next;
            k->next->prev=k->prev;
            free(k);
            return;
        }
    }
}

void searching(int v)
{
    dnode* k=head;

    while(k!=NULL && k->data!=v)
    {

```

```

        k=k->next;
    }
    if(k==NULL)
    {
        cout<<"Not found";
        return;
    }
    else{
        cout<<"The searching value found : "<<k->data;
        cout<<endl;
    }
}
void lastNode()
{
    if(tail != NULL)
    {
        cout<<"The last node value : "<<tail->data;
        cout<<endl;
    }
}
void prevLastNode()
{
    if(tail != NULL && tail->prev != NULL)
    {
        cout<<"The previous last node value : "<<tail->prev->data;
        cout<<endl;
    }
}
void list_size()
{
    dnode * t = head;
    c = 0;
    while(t!=NULL)
    {
        t=t->next;
        c++;
    }
    cout<<"List size : "<<c<<endl;
}
void reversePrint()
{
    printB();
}
int main()
{
    int n,m,l,p,data,value;
    cout<<"Enter the number of elements that insert at first : ";
    cin>>n;
    cout<<"Enter the elements "<<endl;
    for(int i=0; i<n; i++)
    {

```

```

        int x;
        cin>>x;
        insertFirst(x);
    }
    cout<<"The elements in the forward direction are"<<endl;
    printF();
    cout<<"The elements in the backward direction are"<<endl;
    printB();
    cout<<endl;
    cout<<"Enter the number of elements to insert at last"<<endl;
    cin>>m;
    cout<<"Enter the elements "<<endl;
    for(int i=0; i<m; i++)
    {
        int x;
        cin>>x;
        insertLast(x);
    }
    cout<<"After insert at last the elements in the forward direction
are"<<endl;
    printF();
    cout<<"After insert at last the elements in the backward direction
are"<<endl;
    printB();
    cout<<"Enter a position that insert : ";
    cin>>p;
    cout<<endl;
    cout<<"Enter the value : ";
    cin>>data;
    insertAtPos(data,p);
    cout<<"After insert at position the elements in the forward direction
are"<<endl;
    printF();
    cout<<"After insert at position the elements in the backward direction
are"<<endl;
    printB();
    deleteFirst();
    cout<<"The elements in the forward direction after deleting the first
element are"<<endl;
    printF();
    cout<<"The elements in the backward direction after deleting the first
element are"<<endl;
    printB();
    deleteLast();
    cout<<"The elements in the forward direction after deleting the last
element are"<<endl;
    printF();
    cout<<"The elements in the backward direction after deleting the last
element are"<<endl;
    printB();
    cout<<"Enter the position to delete"<<endl;

```



```

        cin>>p;
        deleteByPos(p);
        cout<<"The elements in the forward direction after deleting the element
at position "<<p<<" are"<<endl;
        printF();
        cout<<"The elements in the backward direction after deleting the
element at position "<<p<<" are"<<endl;
        printB();
        cout<<"Enter the value to delete"<<endl;
        cin>>l;
        deleteByValue(l);
        cout<<"The elements in the forward direction after deleting the element
with value "<<l<<" are"<<endl;
        printF();
        cout<<"The elements in the backward direction after deleting the
element with value "<<l<<" are"<<endl;
        printB();
        cout<<"Searching value : ";
        cin>>value;
        searching(value);
        lastNode();
        prevLastNode();
        list_size();
        reversePrint();
}

```

Execution Trace (Input & Output): *(User input is highlighted in bold blue)*

Enter the number of elements that insert at first : **2**

Enter the elements

10 20

The elements in the forward direction are

20 10

The elements in the backward direction are

10 20

Enter the number of elements to insert at last

2

Enter the elements

30 40

After insert at last the elements in the forward direction are

20 10 30 40

After insert at last the elements in the backward direction are

40 30 10 20

Enter a position that insert : **2**

Enter the value : **25**

After insert at position the elements in the forward direction are

20 25 10 30 40

After insert at position the elements in the backward direction are

40 30 10 25 20

The elements in the forward direction after deleting the first element are

25 10 30 40

The elements in the backward direction after deleting the first element are

40 30 10 25

The elements in the forward direction after deleting the last element are

25 10 30

The elements in the backward direction after deleting the last element are

30 10 25

Enter the position to delete

2

The elements in the forward direction after deleting the element at position 2 are

25 30

The elements in the backward direction after deleting the element at position 2 are

30 25

Enter the value to delete

30

The elements in the forward direction after deleting the element with value 30 are

25

The elements in the backward direction after deleting the element with value 30 are

25

Searching value : **25**

The searching value found : 25

The last node value : 25

List size : 1

Reverse print of list : 25